

ANALYSIS AND REPORTING

Homework #2

Shawn Tribuce

postTweet per second	668
getTimeline per second	588

postTweet dropped from 795/sec (MySQL) to 668/sec (Redis) due to the bucketing issue

getTimeline jumped from 3.47/sec (MySQL) to 588/sec (Redis) because no JOIN is needed

Observations: I chose to use method 1 to create buckets for each follower and distribute the tweets posted by a user to their followers bucket based on their timeline respectively. For my storage I facilitated 3 main methods closely resembling the ones we performed in class:

Hash Map: tweet: {tweet_id} - This was used to store the tweet data for each tweet keyed by a unique auto-incremented tweet ID.

List: timeline: {user_id} - Stored the tweet IDs for each user home timeline with the most recent tweet ID pushed to front using lpush. I decided to cap it at 1000 entries to manage memory.

Set: followers: {user_id} - Which stores all follower IDs for a given user and distributes tweets to each followers' buckets while managing duplicates.

Advantages: getTimeline is extremely fast since the timeline is pre-build with no delay between posting tweets and it appearing on the followers' timeline.

Disadvantages: postTweet gets slower as a user gains more followers since the fan out loop grows, so users with a large following will trigger a lot more writes weakening the performance/speed.

AI Usage

I used Claude to assist me with any errors I had when trying to run my script including debugging, in addition to writing multiple comments in my code with co-pilot. I ensured to revise any input it had given me to ensure it was my work.